ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES MÓVILES

TALLER 1: PROMPTS Y RESPUESTAS

Integrantes:

Dayanna Bravo

Esteban Gómez

Andreina Pallo

DOCENTE:

Vicente Eguez Sarzosa

CURSO:

GR1SW

FECHA DE ENTREGA: 15-04-2026

Quito - Pichincha - Ecuador

2026 A

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

- **Prompt**:

"Estoy haciendo un taller sobre aplicaciones móviles y necesito entender qué es el desarrollo nativo puro para Android e iOS. ¿Me puedes explicar cómo han cambiado los lenguajes y las herramientas desde que empezaron hasta lo que usamos hoy en 2026? También me interesa saber por qué se dice que el nativo sigue siendo la mejor opción para el rendimiento."

Respuesta:

El desarrollo nativo consiste en crear aplicaciones utilizando los lenguajes y herramientas oficiales de los dueños de cada plataforma (Apple y Google). Esto garantiza que el código hable directamente con el hardware del teléfono sin intermediarios.

- En el lado de Android: Todo empezó en 2008 usando Java y el entorno Eclipse. Con el tiempo, Google lanzó Android Studio y cambió la forma en que el código se ejecuta con ART (Android Runtime). El gran salto ocurrió con la llegada de Kotlin, que en 2026 ya está en su versión 2.0. Esta versión es mucho más rápida gracias al compilador K2 y permite crear interfaces modernas con Jetpack Compose de forma más eficiente.
- En el lado de iOS: Al principio se usaba Objective-C, un lenguaje potente pero complejo de gestionar. En 2014, Apple lanzó Swift para facilitar el desarrollo. Para 2026, con Swift 6.0, el sistema es extremadamente seguro y evita errores cuando la app intenta hacer varias tareas al mismo tiempo.

El desarrollo nativo sigue siendo el estándar de oro porque permite aprovechar al máximo la batería y el procesador. Además, cuando sale un nuevo sensor o función en un teléfono, los lenguajes nativos son los primeros en poder usarlo sin esperar a que otros frameworks se actualicen.

- **Prompt:**

"He estado leyendo sobre aplicaciones que se hacen con tecnologías web como PhoneGap o el primer Ionic. ¿Cómo funcionan estas apps por dentro técnicamente? ¿Por qué fueron tan importantes hace unos años y por qué ya no se usan tanto para aplicaciones grandes?"

Respuesta:

Las aplicaciones híbridas o basadas en WebView surgieron para que los desarrolladores web pudieran crear apps móviles sin tener que aprender lenguajes nuevos. Su arquitectura se basa en "envolver" una página web dentro de una aplicación móvil.

- Funcionamiento técnico: La aplicación es en realidad un navegador interno (WebView) que ocupa toda la pantalla. Para que el JavaScript de la web pudiera usar la cámara o el GPS, se crearon unos "plugins" que hacían de puente. El código web enviaba una orden, el plugin la traducía a código nativo y el teléfono respondía.
- Auge y caída: Fue una tecnología revolucionaria entre 2011 y 2015 porque permitía ahorrar mucho dinero al usar un solo código para Android e iOS. Sin embargo, el problema principal era la velocidad. Como todo corre dentro de un navegador, las animaciones y el movimiento de las listas suelen tener retraso o "lag". En 2026, esta tecnología ha quedado relegada a aplicaciones muy sencillas o de uso interno en empresas, ya que no pueden competir con la fluidez que exigen los usuarios actuales.

- **Prompt**:

"¿Cómo funcionan exactamente frameworks como React Native o NativeScript? Sé que usan JavaScript pero no son iguales a los híbridos viejos. ¿Qué es ese famoso 'puente' del que hablan y cómo ha cambiado últimamente para que las apps sean más rápidas?"

Respuesta:

Esta arquitectura dio un paso adelante respecto a los híbridos: en lugar de mostrar una web, el código JavaScript le da órdenes al teléfono para que este dibuje botones y menús reales de Android o iOS.

Evolución del Puente:

- Titanium (2008-2011): Fue el pionero en usar "proxies" para conectar JavaScript con el sistema, aunque consumía mucha memoria.
- NativeScript (2014): Permite llamar a cualquier función del teléfono directamente desde JS sin necesidad de plugins, lo cual es muy potente pero difícil de aprender.
- React Native (2015): Popularizó el uso de un "Bridge" o puente que enviaba mensajes en formato JSON entre el código JS y el sistema nativo.

Estado en 2026: El problema del puente JSON es que era lento para animaciones complejas. Por eso, en los últimos años se implementó la JSI (JavaScript Interface). Esto eliminó el envío de mensajes de texto lentos y permitió que JavaScript se comunique directamente con el motor nativo en C++. Gracias a esto, las apps actuales hechas con React Native funcionan casi tan rápido como las nativas.

- **Prompt**:

"Respecto a Flutter porque dicen que 'dibuja' su propia interfaz en lugar de usar la del teléfono. ¿Qué significa eso técnicamente? ¿Cómo funciona su motor gráfico y por qué dicen que es más fluido que otras opciones?"

Respuesta:

Flutter, desarrollado por Google, rompió con el modelo tradicional. En lugar de pedirle al sistema operativo que ponga un botón, Flutter pide un espacio en blanco y él mismo dibuja el botón píxel por píxel.

- El Motor Gráfico: Flutter utiliza un motor llamado Impeller (que reemplazó al antiguo Skia). Impeller habla directamente con la tarjeta gráfica (GPU) del teléfono usando lenguajes de bajo nivel como Metal en iOS o Vulkan en Android. Esto permite que la aplicación mantenga una velocidad constante de 60 o 120 cuadros por segundo sin importar qué tan vieja sea la versión del sistema operativo.
- Compilación AOT: A diferencia de otras tecnologías que interpretan el código mientras la app corre, el lenguaje de Flutter (Dart) se compila a código máquina real antes de que instales la app. Esto hace que el inicio de la aplicación sea casi instantáneo.
- Ventaja competitiva: En 2026, es la opción favorita para apps que necesitan un diseño muy personalizado o idéntico en todos los dispositivos, ya que el framework tiene el control total de lo que se ve en pantalla.

- **Prompt**:

"Todo está cambiando con las pantallas plegables, las gafas de realidad aumentada y los relojes. ¿Cuáles son los retos técnicos más grandes aquí? ¿Por qué es más difícil hacer una app para un teléfono que se dobla o para AR que para una pantalla normal?"

Respuesta:

El futuro del desarrollo móvil obliga a las aplicaciones a salir del "rectángulo plano" y adaptarse al espacio físico y a dispositivos con formas cambiantes.

- Dispositivos Plegables (Foldables): El gran reto es la continuidad. Cuando doblas o desdoblas el teléfono, la aplicación no puede reiniciarse ni perder lo que el usuario estaba haciendo. Técnicamente, esto requiere arquitecturas reactivas que puedan redibujar toda la interfaz en milisegundos sin perder el "estado" de la app. APIs como Jetpack WindowManager son las que permiten que el software sepa en qué ángulo está la pantalla.
- Realidad Aumentada (AR/XR): Aquí el enemigo es la latencia. Si mueves la cabeza y el objeto virtual tarda en moverse, el usuario se marea. Se requieren cálculos matemáticos muy rápidos (matrices de transformación) que deben hacerse directamente en el procesador gráfico. Por esta razón, para 2026, la mayoría de las

experiencias de AR se siguen haciendo con código nativo o motores 3D especializados como Unity.

- Wearables: En relojes y anillos inteligentes, el reto es el consumo de energía. La arquitectura de estas apps debe ser mínima, despertando al procesador solo lo justo para mostrar información y volviendo a dormirlo para no agotar la batería.

En conclusión, para 2026, la arquitectura de una app ya no solo gestiona datos, sino que debe gestionar el espacio físico y la adaptación constante al hardware.

Parte 2: Análisis Crítico de Aplicaciones

1. Instagram

   Prompt: Necesito elaborar el análisis de la app de Instagram para un taller, necesito saber técnicamente por qué es mejor que sea nativa para manejar el funcionamiento de la aplicación en tiempo real comparado con algo como Flutter. También, dime qué problemas tendría si intentara procesar tantos datos de tráfico con un 'puente' tipo React Native y cómo se adaptaría Instagram si usara animaciones fluidas. Dame una respuesta con base en ingeniería de software.

Instagram es una aplicación con altísima exigencia en rendimiento en tiempo real, especialmente en:

- Scroll infinito con contenido multimedia
- Reproducción de video (Reels, Stories)
- Interacciones en vivo (likes, comentarios)
- Uso intensivo de cámara y procesamiento de imágenes

Desde una perspectiva de ingeniería:

- Control total del hardware:

  El desarrollo nativo permite acceso directo a GPU, cámara, sensores y optimización de memoria.

- Menor latencia:
  Evita capas intermedias como el "puente" de React Native o el motor intermedio de Flutter.
- Optimización específica por plataforma:
  Instagram puede adaptar el comportamiento según iOS o Android, maximizando rendimiento en cada uno.
- Escalabilidad del rendimiento:
  Al manejar millones de usuarios concurrentes, pequeñas mejoras a nivel nativo tienen gran impacto.

Instagram sí ha usado React Native en partes, pero no como base total.

Razones técnicas:

- Limitaciones del puente (React Native):
  - Comunicación JS ↔ nativo puede generar latencia.
  - No es ideal para animaciones complejas o video en tiempo real.
- Flutter (motor propio):
  - Introduce una capa adicional de renderizado.
  - Puede dificultar integración profunda con APIs nativas críticas.
- Costo de migración:
  - Reescribir una app tan grande implica riesgo operativo alto.

Los principales desafios que presenta instagram son:

Principales desafíos:

- Renderizado eficiente de listas enormes (feeds):
  - Uso de técnicas como *virtualización de listas*.
  - Carga diferida (lazy loading).
- Procesamiento de video en tiempo real:
  - Requiere uso intensivo de GPU.
  - Compresión, buffering y sincronización.

- Animaciones fluidas (60 FPS o más):
  - Transiciones entre historias, reels, etc.
  - Necesidad de minimizar bloqueos del hilo principal.
- Gestión de memoria:
  - Evitar leaks al cargar imágenes/videos constantemente.

Si Instagram fuera 100% React Native:

- Problema principal: el bridge (JS ↔ nativo)

Cuellos de botella:

- Scroll de miles de publicaciones:
  - Cada interacción pasa por el bridge → posible lag
- Animaciones complejas:
  - No siempre se ejecutan en el hilo nativo → pérdida de FPS
- Eventos de alta frecuencia:
  - Likes, comentarios en tiempo real → saturación del bridge

2. Waze:

- **Promt:**

  Hola, ayúdame con el análisis de la app de waze para un taller de la u. Necesito saber técnicamente por qué es mejor que sea nativa para manejar mapas 3D y el GPS en tiempo real comparado con algo como Flutter. También, dime qué problemas tendría si intentara procesar tantos datos de tráfico con un 'puente' tipo React Native y cómo se adaptaría Waze si salieran lentes de Realidad Aumentada. Dame una respuesta con base en ingeniería de software.

- **Respuesta:**

Análisis de Rendimiento y Arquitectura: Caso Waze

- **Desafíos arquitectónicos (Renderizado y Seguridad):** A diferencia de Nubank (que usa Flutter), Waze no puede permitirse delegar todo el renderizado a un framework de alto nivel. Su principal desafío es el **renderizado de mapas dinámicos**. Waze utiliza motores gráficos optimizados (muchas veces escritos en **C++** a través de NDK en Android) para dibujar la cartografía en 3D. Esto le permite manejar animaciones complejas de tráfico y rutas sin sobrecargar la CPU, algo que en frameworks como Flutter podría causar "jank" (pequeños saltos en la imagen) debido a la capa de abstracción del motor Skia/Impeller al procesar tantos vectores geográficos en tiempo real.
- **El cuello de botella del "Puente" y los 60 FPS:** En aplicaciones como Discord (React Native), el puente (Bridge) es el problema. En Waze, el cuello de botella es el **paso de datos entre el sensor GPS/Red y la capa de visualización**. Para lograr un scroll de mapa fluido a 60 FPS, Waze minimiza el uso de puentes costosos, procesando la lógica de posicionamiento de forma asíncrona. Si Waze fuera puramente React Native, el envío constante de coordenadas GPS a través del puente de JavaScript saturaría la comunicación, haciendo que el mapa se sienta "lento" o pesado.

El Desafío de las Nuevas Pantallas (Realidad Aumentada)

Si Apple o Samsung lanzan lentes de AR, la arquitectura de **Waze tendría una ventaja competitiva**. Al estar construida con una base sólida de código nativo y motores de bajo nivel (C++), tiene acceso directo a las APIs de procesamiento visual y sensores de profundidad.

- **React Native/Flutter:** Tendrían dificultades porque dependen de que el framework cree un "wrapper" para esas nuevas tecnologías.
- **Waze:** Podría implementar "Head-up Display" (HUD) directamente sobre el hardware de los lentes con bajísima latencia, algo vital para que el usuario no sufra mareos al ver las flechas de navegación sobre la calle real.

- ¿Qué es un aplicativo nativo, hay varios, como se clasifican?

Un aplicativo nativo es software construido específicamente para el "lenguaje materno" del sistema operativo. La clave de la ingeniería aquí es que no hay intermediarios que traduzcan las instrucciones en tiempo real; el código habla directo con el procesador.

Se clasifican así según cómo manejan el hardware:

- **Nativo Puro:** Es el estándar. Usas Kotlin (Android) o Swift (iOS). Es el más estable porque usa las herramientas oficiales de Google o Apple.
- **Nativo de Alto Rendimiento (NDK/C++):** Aquí entran apps como **Waze** o juegos pesados. Usan C++ para tareas matemáticas brutas (como calcular rutas o renderizar mapas 3D) y solo usan el lenguaje nativo (Kotlin/Swift) para la "envoltura" o interfaz. Es nativo, pero de bajo nivel.
- **Nativo Multiplataforma:** Aquí hay confusión. **Flutter** se considera nativo porque *compila* a código de máquina, pero no usa los botones o menús originales del sistema, sino que los "dibuja". **React Native**, en cambio, sí usa los botones originales pero los controla desde un hilo de JavaScript. Ambos intentan ser nativos pero desde un solo código base.